

PPO (Proximal Policy Optimization)

First, a story

You are a martial artist. After each bout (one rollout), the coach (the advantage function $\hat{A}$) hands you a detailed review: “the side kick at second 23 should be faster”, “the defense at second 47 was too conservative”. The report was written **based on how you fought then** (the old policy $\pi_{\theta_{\text{old}}}$).

You take the report and adjust toward the optimum. It doesn’t feel like enough, so you adjust again... after $K = 20$ rounds you have become a completely different fighter — yet the coach’s advice is still that original report, written about the you of twenty rounds ago. The more you follow it, the stranger your movements become, until you lose disastrously.

The core conflict:

- the data was collected by the old policy $\pi_{\theta_{\text{old}}}$;
- each round’s update optimizes the new policy π_{θ} ;
- the more updates, the further apart they drift — the staler the data.

This is the dilemma of unconstrained multi-round policy updates, and the core problem PPO solves:

How to update the policy many times, safely, without wasting samples?

Formalization

The policy gradient theorem gives the gradient direction:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]$$

To reuse data collected by the old policy, introduce the **importance-sampling weight** (the probability ratio):

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

$r_t = 1$ means the new and old policies agree completely at this step; $r_t \gg 1$ or $r_t \ll 1$ means the policy has drifted severely.

Substituting into the surrogate objective:

$$L^{\text{CPI}}(\theta) = \hat{\mathbb{E}}_t \left[r_t(\theta) \hat{A}_t \right]$$

In plain words: multiply each step’s advantage $\hat{A}_t$ by “the ratio of new to old policy”, and optimizing this surrogate repeatedly lets one batch of data be reused many times. **The problem:** after K rounds r_t can drift far from 1 — the surrogate becomes untrustworthy and the gradient direction fails.

Why does unconstrained updating collapse?

The valid range of importance sampling

Importance sampling implicitly assumes the two distributions are “close enough”. If π_{θ} and $\pi_{\theta_{\text{old}}}$ diverge too much, the estimator’s variance explodes:

$$\text{Var} \left[r_t(\theta) \hat{A}_t \right] \propto \mathbb{E} \left[r_t^2(\theta) \right] - 1$$

As r_t departs from 1 (in either direction), the gradient estimate’s variance grows sharply and the update direction becomes unreliable.

K rounds of updating drift r

Take clip-free PPO ($K = 20$ rounds): after each round θ changes and $r_t(\theta)$ drifts along:

- round 1: $r_t \approx 1$, the gradient estimate is trustworthy;
- round 4: r_t starts to drift; the advantage estimate becomes biased;
- round 8: r_t is severely off, the advantage estimate fails entirely, the gradient direction is wrong — training collapses.

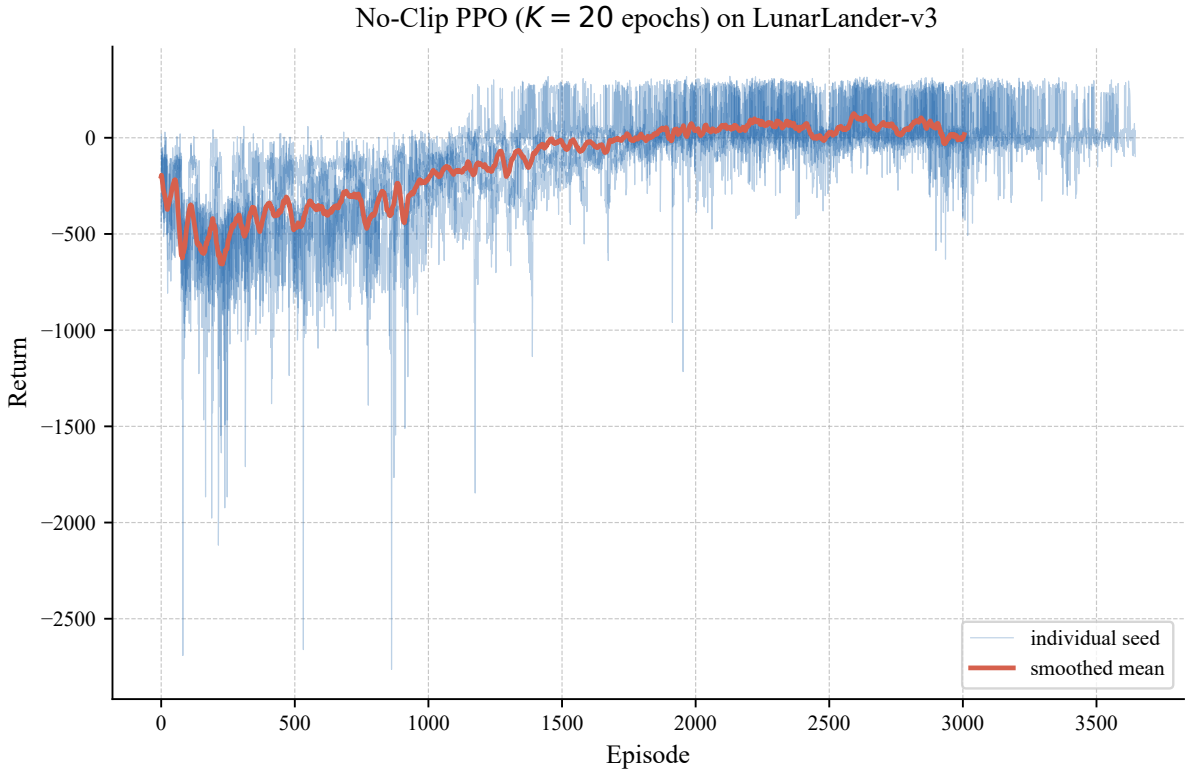


Figure 1: Clip-free PPO ($K = 20$ rounds, mini-batch) on LunarLander-v3, 5 seeds (thin blue lines). Some seeds’ returns drop to -8000 early on (40 times worse than the random policy), then recover slowly. The smoothed mean (red) stays below the clipped version throughout training.

PPO-Clip: the truncated surrogate objective

The core formula

PPO’s solution is remarkably simple: **clip r_t to the neighborhood of $[1 - \varepsilon, 1 + \varepsilon]$** ; beyond the bound, no gradient incentive remains.

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

ε is typically 0.2.

Intuition: cutting off both extremes

Read min in two cases.

Case A: $\hat{A}_t > 0$ (this action beat the average; raise its probability)

- r_t rising (the new policy favors this action) $\rightarrow$ good, but past $1 + \varepsilon$ the clipped value is used and the gradient goes to zero;

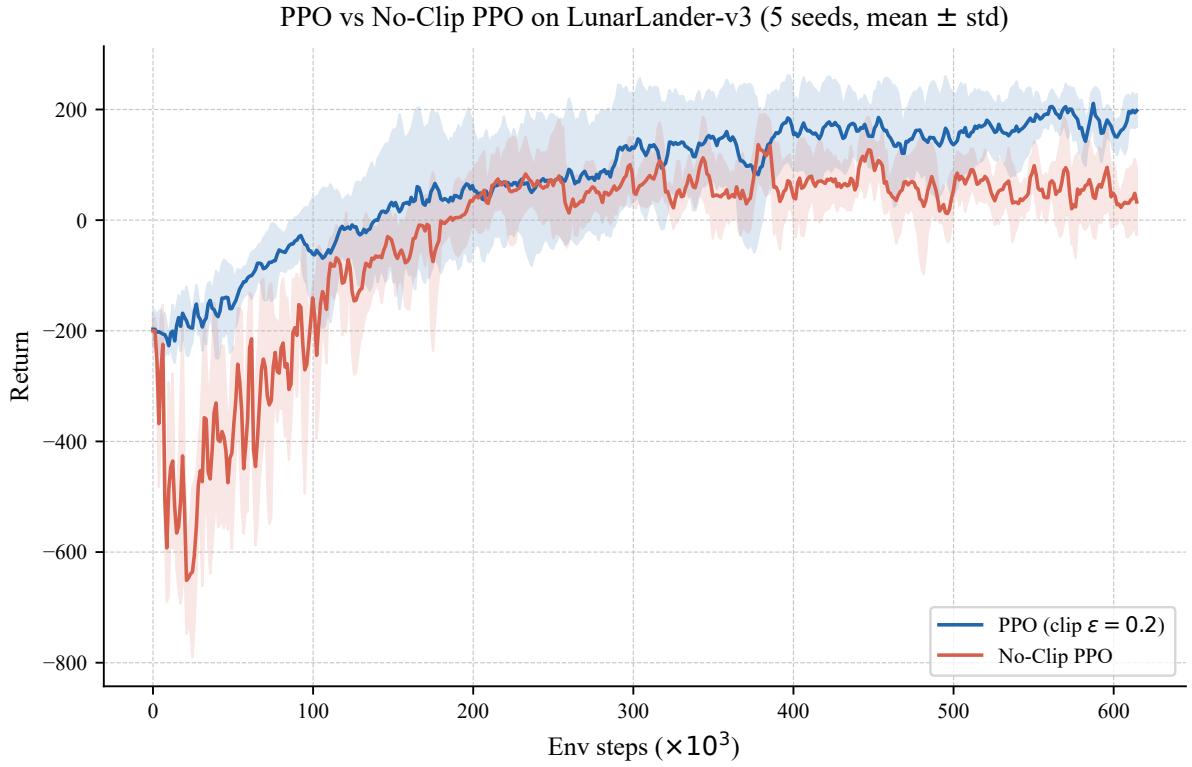


Figure 2: PPO-Clip (blue) vs. clip-free PPO (red) on LunarLander-v3 (5 seeds, mean $\pm$ std). The x-axis is environment steps ($\times 10^3$); both methods use exactly the same total steps ($300 \times 2048 \approx 614k$), ruling out the confound that “a better policy has longer episodes and hence fewer episodes”. The clipped version rises steadily, entering positive returns after about 400k steps; the clip-free version crashes to $-600 \sim -1000$ early, recovers slowly, and never catches up, with larger variance.

- **Core principle:** the policy change is “already enough” — no further pushing.

Case B: $\hat{A}_t < 0$ (this action was worse; lower its probability)

- r_t falling (the new policy avoids this action) $\rightarrow$ good, but below $1 - \varepsilon$ the clipped value is used and the gradient goes to zero;
- **Core principle:** as above — excessive avoidance is no longer rewarded either.

Only updates heading in the favorable direction and not yet past the bound provide gradient drive.

How the clip suppresses r ’s drift

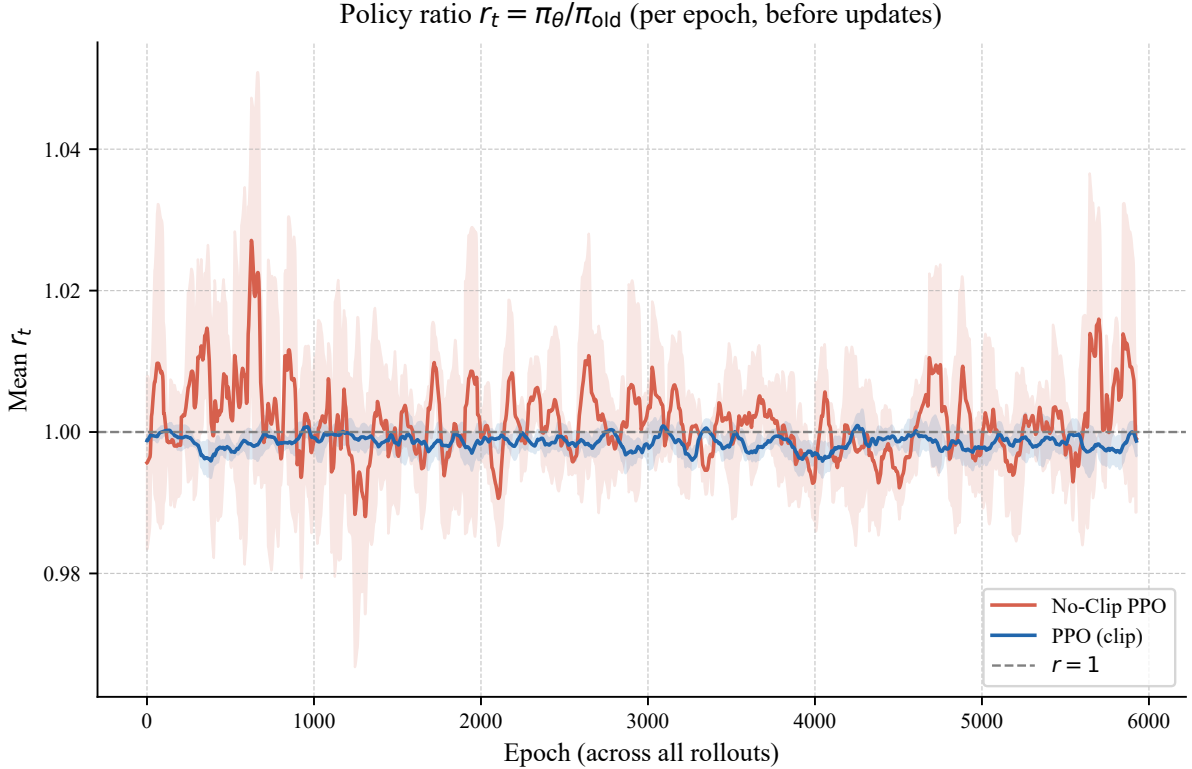


Figure 3: The batch-mean policy ratio r_t , measured on the full batch before each epoch (5 seeds, mean $\pm$ std). Clip-free (red) deviates more, reaching ~ 1.017 early; PPO-Clip (blue) stays near 1. Beyond the mean, individual transitions’ r_t can far exceed the 1.2 clip bound — it is exactly these tails being clipped that prevents the accumulation of erroneous gradients.

Why does the clip work?

1. **Suppresses the “step size” of each update:** once r_t leaves $[1 - \varepsilon, 1 + \varepsilon]$, that sample no longer contributes gradient in the direction of further deviation — drift is **softly suppressed**, not hard-forbidden; individual r_t may still exceed the bound (see Figure 3);
2. **Symmetric protection:** an action’s probability can be neither over-increased nor over-decreased;
3. **A pessimistic lower bound:** L^{CLIP} lower-bounds L^{CPI} and never overstates the improvement — updates happen only when the improvement is real.

Generalized Advantage Estimation (GAE)

Why GAE

The advantage function $A(s, a) = Q(s, a) - V(s)$ measures “how much better than average”. Practice offers two extreme estimators:

Estimator	Bias	Variance
$\hat{A}^{\text{MC}} = \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V(s_t)$	zero	very large
$\hat{A}^{\text{TD}} = r_t + \gamma V(s_{t+1}) - V(s_t)$	biased	very small

GAE interpolates between the two, with $\lambda \in [0, 1]$ tuning the bias–variance trade-off.

The GAE formula

Define the TD residual (one-step temporal-difference error):

$$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE sums exponentially weighted multi-step TD residuals:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V$$

The equivalent recursion (used in implementations, inducting backwards from the end):

$$\hat{A}_t = \delta_t^V + \gamma \lambda \hat{A}_{t+1}$$

The meaning of λ : interpolating between TD and MC

λ	degenerates to	effect
$\lambda = 0$	$\hat{A}_t = \delta_t^V$ (1-step TD)	low variance, biased
$\lambda = 1$	$\hat{A}_t = \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V(s_t)$ (MC)	zero bias, high variance
$\lambda = 0.95$	common in practice	bias/variance compromise

In plain words: the larger λ , the more one trusts the actually sampled distant returns rather than the critic’s predictions. PPO typically takes $\gamma = 0.99$, $\lambda = 0.95$.

The full PPO algorithm

The total loss

$$L^{\text{PPO}}(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 S[\pi_\theta]$$

The terms:

- L^{CLIP} : the clipped policy-gradient loss (maximized);
- $L^{\text{VF}} = \hat{\mathbb{E}}_t \left[\left(V_\theta(s_t) - \hat{V}_t^{\text{target}} \right)^2 \right]$: the critic’s squared error (minimized, $c_1 \approx 0.5$);
- $S[\pi_\theta] = \hat{\mathbb{E}}_t [-\sum_a \pi_\theta(a|s_t) \log \pi_\theta(a|s_t)]$: the entropy bonus preventing premature policy degeneration (maximized, $c_2 \approx 0.01$).

The actor and critic **share the network parameters** θ ; one backpropagation updates both.

The PPO-Clip loop

1. **Collect a rollout**: run T steps in the environment with $\pi_{\theta_{\text{old}}}$, recording (s_t, a_t, r_t, s_{t+1}) ;
2. **Estimate advantages**: compute $\hat{A}_t$ by GAE; value targets $\hat{V}_t^{\text{target}} = \hat{A}_t + V_{\theta_{\text{old}}}(s_t)$;
3. **K epochs of mini-batch updates**:
 - (a) forward pass, compute $r_t(\theta)$ and $L^{\text{PPO}}(\theta)$;
 - (b) clip gradients ($\|\nabla\theta\|_2 \leq 0.5$), backpropagate;
 - (c) update θ .
4. Set $\theta_{\text{old}} \leftarrow \theta$, return to step 1.

Why are K epochs safe now? Once a sample’s r_t leaves $[1 - \varepsilon, 1 + \varepsilon]$, the clip removes its gradient in the direction of further deviation, suppressing the error accumulation of K epochs on the same batch. Note this is a **soft constraint**: individual r_t still exceed the bound (Figure 3) — there is simply no gradient incentive in the deviating direction to amplify them further.

PPO vs other policy-gradient methods

One table to compare

Method	Sample efficiency	Stability	Complexity
REINFORCE	very low (one update per batch)	low (high variance)	simplest
A2C	low (same)	medium	simple
TRPO	high (one natural-gradient update per batch)	high (KL constraint)	very complex (second-order)
PPO-Clip	high (K epochs per batch)	high (clip constraint)	simple (first-order)

Why is PPO more popular than TRPO?

TRPO (Trust Region Policy Optimization) uses the KL divergence as a hard constraint:

$$\max_{\theta} L^{\text{CPI}}(\theta) \quad \text{s.t.} \quad \hat{\mathbb{E}}_t[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | s_t) \parallel \pi_{\theta}(\cdot | s_t))] \leq \delta$$

Theoretically the strictest — but it requires the natural gradient (inverting the Fisher information matrix), prohibitively expensive on deep networks.

PPO turns the “trust region” into a **soft constraint**: beyond the bound nothing is penalized; it simply receives no further gradient incentive.

- **TRPO**: “cross the line and get punished” (hard constraint, second-order optimization);
- **PPO**: “near the line, we stop pushing” (soft constraint, first-order optimization).

Comparable results — but PPO needs only first-order gradients, is simple and fast, and has become the default baseline among on-policy methods.

Where this chapter sits in the RL landscape

1. **The TD route** $\rightarrow$ Q-Learning $\rightarrow$ DQN (discrete actions, value-based);
2. **The MC route** $\rightarrow$ REINFORCE $\rightarrow$ policy gradients (high variance, one update per batch);
3. **Actor-Critic** $\rightarrow$ GAE $\rightarrow$ PPO (this chapter: low variance + safe multi-epoch updates);
4. **Continuous actions / offline settings** $\rightarrow$ SAC (maximum-entropy RL) $\rightarrow$ offline RL (IQL etc.).

PPO is today’s default on-policy baseline — nearly every RLHF pipeline (LLM alignment) builds on PPO at its core. Given how robust PPO already is, why do continuous-action and offline settings still need their own algorithms?

References

1. Schulman et al. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.
2. Schulman et al. (2015). Trust Region Policy Optimization. *ICML*.
3. Schulman et al. (2016). High-Dimensional Continuous Control Using Generalized Advantage Estimation. *ICLR*.
4. Sutton & Barto (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Chapter 13.
5. Hands-on RL (Chinese). Chapter 12: the PPO algorithm.